

BOWIE STATE UNIVERSITY

Department of Computer Science • College of Arts and Sciences

COSC 113: Computer Science II

Final Group Project — Spring 2026

Instructor: Devharsh Trivedi, Ph.D., CISSP	Email: dtrivedi@bowiestate.edu
Office: Computer Science Building 221	Office Hours: TWR 1:30 – 3:40 PM
Duration: 3/24/2026 – 5/7/2026 (7 weeks)	Presentation: 5/12/2026 (during class)

Objective

This project requires teams of **2 – 4 students** to develop a functional and thoroughly tested application (console or simple GUI). The deliverable must showcase advanced Java programming skills and an understanding of professional, collaborative development workflows. **You cannot work solo.** All members must contribute equally to the project.

Team Formation

You must accept the group assignment from the GitHub Classroom link provided. One member can create a new team, and the remaining members join that team. Be sure to select the correct team — **you will not be able to change this later.** After accepting the assignment, check your email for access permissions to the GitHub repository.

Important: Commit all your code, UML diagrams, and presentation slides to the GitHub repository. **Do not submit anything on Blackboard.**

Technology Requirements

- **Java Version:** The project must use Java 17 (LTS) or a more recent version, leveraging modern Java language features (e.g., records, switch expressions, text blocks) where appropriate.
- **Version Control:** GitHub must be used for version control. Strict adherence to the GitHub Flow branching model is required, including the use of feature branches, regular commits, and Pull Requests (PRs) with basic peer code review before merging to the main branch. Quality of commits, branch management, and commit messages will be graded (10 pts).
- **Collaborative Contribution:** The final repository history must clearly show commits from all group members, demonstrating equitable distribution of work. GitHub contribution from all team members is part of the grading criteria (10 pts).

- **Dependency Management:** The project should be structured with a build automation tool. Maven or Gradle is recommended to manage dependencies (e.g., for testing frameworks or external libraries).
- **Testing and Quality Assurance** (Optional, but Strongly Encouraged): Teams are highly encouraged to implement Unit Tests using JUnit 5. If you choose to include testing, it must demonstrate rigor by achieving a minimum of 70% statement coverage for your core business logic. Coverage must be verified and reported using an industry-standard tool like JaCoCo.

Architecture Requirements

Object-Oriented Programming (OOP): The design must clearly demonstrate the application of all four core OOP principles — each graded separately (10 pts each, 40 pts total):

- **Abstraction** in design and implementation.
 - **Encapsulation** (e.g., proper use of private fields and getters/setters).
 - **Inheritance** (e.g., classes extending a base class or interface).
 - **Polymorphism** (e.g., method overriding or interface usage).
- **Data Structures:** The implementation must utilize an effective advanced data structure (e.g., TreeMap, LinkedList, PriorityQueue) or a custom implementation of a Heap, Stack, or Queue. The accompanying report should explain the time complexity justification for its use (10 pts).

Deliverables

1. Code Report and Documentation — A comprehensive report detailing:

- **Architecture & Design:** Clear, concise UML Class Diagrams (graded for inclusion and correctness, 10 pts) and an explanation of the chosen Design Pattern.
- **Data Structure Justification:** Explanation of the advanced data structure(s) and their benefits.
- **Contribution Log:** Explicit delineation of specific contributions from each team member, linking to relevant GitHub PRs/commits.

2. Professional Presentation (5/12/2026) — 10 minutes for the presentation + 2 minutes Q&A. All team members must present and give a live working demo of their part. You must demonstrate your UML class diagrams and show commit history from all members. Graded on clarity, professionalism, and thoroughness (10 pts).

3. Advanced Java Mastery — Implementation must demonstrate integration of advanced concepts such as recursion, file management, database integration, lambda expressions, or regex (10 pts). Also includes robust use of File I/O (e.g., reading/writing data to CSV, JSON, or a flat file for persistence) and Exception Handling (e.g., custom exceptions).

4. GitHub Repository — All code, diagrams, and slides must be committed to the GitHub repository. Nothing should be submitted on Blackboard. Quality of commits, branch management, and commit messages will be evaluated (10 pts), along with equitable contribution from all members (10 pts).

Presentation Requirements

- Each team will have **10 minutes** for the presentation and **2 minutes** for the Q&A session.
- Teams cannot have more than 4 students and you cannot work alone.
- All students of a team must be present and give a **live demo of their part** of the project.
- GitHub repository must show commits from all members.
- You must present your **UML class diagrams** during the presentation.
- All teams are required to give a **live, working demonstration** of your project using the Java programming language.

Attendance & Submission Options

- **In-Person (Section 001) Students:** You must give your presentation in person during class on 5/12/2026.
- **HyFlex Students (Section 400):** You have two options:
 1. Join the class session to present in person.
 2. Submit a video recording of your presentation beforehand.

Submission Instructions

1. Accept the group assignment from the GitHub Classroom link provided via email/Blackboard.
2. One member creates a team; other members join that same team.
3. Commit all deliverables (code, UML diagrams, presentation slides, report) to the repository.
4. **Do not submit anything on Blackboard** — GitHub is the sole submission platform.
5. Check your email for access permissions after accepting the assignment.

Grading Rubric (Total: 100 Points)

Category	Specific Requirement	Points
OOP Principles	Use of Abstraction in design and implementation.	10
	Use of Encapsulation (e.g., proper use of private fields and getters/setters).	10
	Demonstrable use of Inheritance (e.g., classes extending a base class/interface).	10
	Demonstrable use of Polymorphism (e.g., method overriding or interface usage).	10
Version Control	Quality of GitHub commits, branch management, and commit messages.	10
	GitHub contribution from all team members (demonstrating fair workload).	10
Design & Structure	Inclusion and correctness of UML Diagrams (e.g., class diagrams).	10
	Effective use of an Advanced Data Structure (e.g., TreeMap, LinkedList, etc.).	10
Project Quality	Quality of Presentation and Documentation (clarity, professionalism, thoroughness).	10
	Integration of Advanced Concepts (e.g., recursion, file management, database integration, lambda expressions, regex).	10
TOTAL		100

Sample Project: The Digital Library Manager

The goal of this project is to develop a robust Java application to automate a library's core operations. This includes managing the inventory of resources (books, media), tracking member accounts, and handling loans, returns, and reservations.

How this sample meets each rubric category:

- **Abstraction (10 pts):** A `LibraryItem` interface defines common functionality (`checkout()`, `isAvailable()`) that all resource types must implement.
- **Encapsulation (10 pts):** Book details (title, author, ISBN) are managed via private fields with getters/setters in the `Book` class.

- **Inheritance (10 pts):** Specialized types like ReferenceBook and FictionBook extend a base Book class.
- **Polymorphism (10 pts):** A common method getDisplayDetails() is overridden in subclasses for type-specific formatting.
- **Version Control (20 pts):** GitHub repository uses feature branches (e.g., "add-book," "process-loan") with Pull Requests for code review. All members commit regularly with clear messages.
- **UML Diagrams (10 pts):** Class diagrams visualize the Book hierarchy and relationships between entities.
- **Advanced Data Structure (10 pts):** HashMap for O(1) retrieval by ISBN, with time complexity justification in the report.
- **Presentation & Documentation (10 pts):** Live demo with UML walkthrough and commit history review.
- **Advanced Concepts (10 pts):** File I/O for data persistence (CSV/JSON); custom Exception Handling for invalid member IDs; lambda expressions for filtering overdue books.

Project Ideas

You may choose any topic from the list below, or propose a custom topic approved by the instructor. All projects must meet the complexity and technical requirements described in the rubric.

Core OOP and Management Systems

Project	Description
Library Catalog System	Model Books, Members, and Librarians using abstract classes (Person) and interfaces (Borrowable).
Employee Calculator Payroll	Abstract Employee class with Full-Time/Part-Time subclasses. Implement a Taxable interface.
University Tracker Enrollment	Manage Students, Professors, and Courses with abstract Person class and Gradable interface.
Hospital Manager Operations	Doctors and Patients via Appointments. Schedulable interface; Aggregation with Specialization.

E-Commerce and Composition-Focused Apps

Project	Description
Shopping Cart Simulation	Cart holding Product objects with Composition. Abstract User class (Admin/Customer) and Discountable interface.

Flight Reservation System	Flights and Passengers linked to Reservations. Composition with FlightSeat; Bookable interface.
Online Food Delivery App	Restaurants, Menu Items, and Orders. Deliverable interface; abstract User class for different roles.
Vehicle Rental Service	Vehicle hierarchy (Car, Truck, Bike) from abstract base. Aggregation between Customer and Rental.

Advanced Systems and Simulations

Project	Description
Advanced Banking Simulator	Abstract Account (Savings, Checking). Transaction interface; Composition linking Customer to accounts.
Smart Home Device Controller	Controllable interface, abstract Device class (Light, Thermostat). Composition with Room object.
Social Media Prototype	User, Post, Comment objects. Shareable and Reportable interfaces; nested composition.
Online Quiz/Test System	Abstract Question class (MCQ, True/False). Scorable interface; Aggregation for Test object.

Important Reminders

- **AI Policy:** The use of AI tools is expressly forbidden for all submitted assignments. Violation will be considered a breach of academic integrity and will result in a failing grade for the course.
- **Academic Integrity:** Plagiarism will result in a zero for the assignment. Students must strictly adhere to the university honor code and professional ethics.
- **Late Policy:** Deadlines are firm. Late submissions cannot be accepted. Once solutions are released, no further submissions will be accepted.
- **Equal Contribution:** All team members must contribute meaningfully. The commit history will be reviewed to verify equitable participation (10 pts of the rubric).